

App BiT — Painel de Inclusão Digital

Autor: Paufer — Frontend Principal

Equipa: 48 · Hackathon BiT B2G

Data: Junho 2026

Estado: Semana 1 — Fundação (MVP mockado)

1. Resumo Executivo

O **App BiT** é uma plataforma B2G (Business to Government) de apoio à decisão para gestores públicos. Consolida dados do dataset **Visent CDRView** (concentração de pessoas e cobertura de rede por região) com fontes públicas, permitindo consultas em linguagem natural sobre 5 serviços críticos:

- **Formações** — Programas de formação tech por região
- **Empregabilidade** — Taxa de emprego formal vs concentração
- **Experiências Estruturantes** — Iniciativas sociais e culturais
- **Mentorias** — Programas de mentoria pública
- **Saúde Mental** — Indicadores de saúde mental vs conectividade

O frontend está construído em **React + Tailwind CSS** com mapa interactivo **Leaflet**. Nesta fase (Semana 1), todos os dados são mockados para demonstrar o fluxo completo.

2. Stack Tecnológico

Camada	Tecnologia	Versão	Propósito
Framework UI	React	18.x	Componentes e estado
Estilização	Tailwind CSS	3.x	Utilitários responsivos
Mapa	Leaflet + react-leaflet	1.9 / 4.x	Mapa interactivo OpenStreetMap
Build (futuro)	Vite	5.x	Dev server e bundling
Teste rápido	CDN (unpkg)	—	index.html standalone via browser

Nota: O stack foi escolhido por simplicidade e rapidez de desenvolvimento. React + Tailwind é a combinação mais produtiva para um MVP B2G. O Leaflet foi escolhido por ser gratuito, sem necessidade de API key.

3. Estrutura do Projecto

```
app-bit/  
├─ index.html           ← Versão standalone (CDN) para testes imediatos  
├─ package.json         ← Dependências npm (react, react-dom)  
├─ node_modules/  
├─ src/  
│   └─ components/  
│       └─ DashboardLayout.jsx ← Componente principal (casca)
```

└ SidebarConsulta.jsx	← Barra de consulta + resultados mockados
└ MapaInterativo.jsx	← Mapa Leaflet com círculos interactivos

Atenção: O projecto ainda não tem Vite configurado (npm registry offline). O ficheiro `index.html` carrega React + Leaflet + Tailwind via CDN e funciona abrindo diretamente no browser. Para build futura, é necessário instalar as dependências e usar `npm run dev`.

4. Componentes React

4.1. DashboardLayout (src/components/DashboardLayout.jsx)

Função: Componente casca que orquestra todo o layout.

Estado gerido:

Estado	Tipo	Padrão	Descrição
<code>servicoAtivo</code>	string	<code>'formacoes'</code>	Serviço selecionado nas tabs
<code>regiaoAtiva</code>	string	<code>'todas'</code>	Filtro de região
<code>idioma</code>	string	<code>'pt'</code>	Idioma atual (pt es en)

Estrutura visual:

Header: [B] App BiT – Painel de Inclusão Digital [PT ES EN] [Badge CDRView]	
Tabs: Formações Empregabilidade Exp. Estruturantes Mentorias Saúde Mental	
SidebarConsulta (40% desktop) scroll interno	MapaInterativo (60% desktop) altura total

Props passadas aos filhos:

Filho	Props
SidebarConsulta	<code>servicoAtivo, regiaoAtiva, onRegiaoChange, corBase, idioma</code>
MapaInterativo	<code>servicoAtivo, idioma</code>

4.2. SidebarConsulta (src/components/SidebarConsulta.jsx)

Função: Coluna lateral de consulta. Permite ao gestor selecionar região, fazer perguntas em linguagem natural e visualizar respostas mockadas.

Estados internos:

Estado	Função
--------	--------

query	Texto digitado no input
resultado	Resposta mockada após submissão (null = estado inicial)
carregando	Simula loading de 1.2s antes de mostrar resposta

Mock data por serviço:

Cada um dos 5 serviços tem:

- **3 sugestões** de perguntas (traduzidas em PT/ES/EN)
- **1 resposta_ia** com análise em linguagem natural
- **2 dados** no formato {regiao, valor, fonte}
- **Fontes** associadas

Exemplo (formacoes):

```
{
  resposta_ia: "A Zona Urbana Norte apresenta 85% de concentração juvenil...",
  dados: [
    { regiao: "Zona Urbana Norte", valor: 28, fonte: "Visent CDRView" },
    { regiao: "Distrito Industrial Sul", valor: 72, fonte: "Visent CDRView" }
  ],
  fontes: ["Visent CDRView", "Ministério da Educação"]
}
```

Estados visuais:

- **Aguardando consulta:** dashed border com "?" e texto explicativo
- **Loading:** spinner animado com a cor do serviço ativo
- **Resultado:** bloco de resposta + grid de cards + badges de fontes + botão "Nova Consulta"

4.3. MapaInterativo (src/components/MapaInterativo.jsx)

Função: Mapa OpenStreetMap com círculos sobrepostos representando as regiões analisadas.

Dados mockados (GET /mapa):

Região	Lat	Lng	Cobertura Rede	Concentração	Indicadores
Zona Urbana Norte	-15.195	12.150	30%	85%	emprego, formacao
Distrito Industrial Sul	-15.210	12.165	75%	40%	conectividade

Lógica de cores:

Condição	Cor	Significado
Cobertura < 40% e concentração > 70% OU indicador do serviço em alerta	#E63946	Alerta — Serviço crítico
Cobertura < 60%	#F4A261	Cobertura Parcial

Cobertura ≥ 60%

#06D6A0

Infraestrutura Sólida

Interações:

- **Popup ao clicar no círculo** — mostra nome, cobertura, concentração, valor do indicador e 3 tags de diagnóstico
- **Tooltip fixo (mobile)** — colado ao fundo do ecrã em mobile, centrado em desktop
- **Legenda flutuante** — canto inferior direito com as 3 cores e significados
- **Cores dinâmicas** — mudam conforme o serviço selecionado nas tabs

Dependências: `react-leaflet` e `leaflet` (npm). No `index.html` é usado Leaflet via CDN.

5. Contratos de API (Mockados)

POST /dados

Request:

```
{
  "consulta": "Onde faltam programas de formação para jovens de baixa renda?",
  "filtros": { "regiao": "Luanda", "indicador": "formacao" },
  "idioma": "pt"
}
```

Response:

```
{
  "resposta_ia": "A Zona Urbana Norte apresenta 85% de concentração juvenil...",
  "dados": [
    { "regiao": "Zona Urbana Norte", "valor": 28, "fonte": "Visent CDRView" },
    { "regiao": "Distrito Industrial Sul", "valor": 72, "fonte": "Visent CDRView" }
  ],
  "fontes": ["Visent CDRView", "Ministério da Educação"]
}
```

GET /mapa

Response:

```
{
  "regioes": [
    {
      "regiao": "Zona Urbana Norte",
      "lat": -15.195,
      "lng": 12.150,
      "concentracao": 85,
      "cobertura_rede": 30,
      "indicadores": ["emprego", "formacao"]
    },
    {
      "regiao": "Distrito Industrial Sul",
      "lat": -15.210,
      "lng": 12.165,
      "concentracao": 40,
      "cobertura_rede": 75,
      "indicadores": ["conectividade"]
    }
  ]
}
```

Importante para o backend: O frontend espera exatamente este formato. Qualquer mudança no contrato quebra a exibição dos dados.

6. Internacionalização (PT/ES/EN)

O sistema suporta 3 idiomas: **Português, Espanhol e Inglês**.

Mecanismo:

- Estado `idioma` no `DashboardLayout` (valor padrão: `'pt'`)
- Passado como prop para `SidebarConsulta` e `MapaInterativo`
- Função `t(obj, idioma)` que retorna a string no idioma correto:

```
function t(obj, idioma) {
  return typeof obj === 'string' ? obj : obj[idioma];
}
```

- Selector de idioma no header (PT | ES | EN) — toggle visual com bg branco no ativo

Textos traduzidos:

- Nomes dos 5 serviços
- Sugestões de perguntas (3 por serviço × 3 idiomas = 45 frases)
- Respostas da IA (5 serviços × 3 idiomas = 15 blocos de texto)
- Nomes das regiões
- UI genérica: placeholders, botões, legendas, tooltips, labels
- Legenda do mapa e status de alerta
- Badge do dataset
- Nome do dashboard

Como adicionar um novo idioma: Acrescentar a chave nos objetos de tradução (ex: `fr: '...'`), adicionar o botão no seletor e o caso no `idioma ===`.

7. Estratégia Responsiva

O layout adapta-se a 3 breakpoints usando classes utilitárias do Tailwind:

Breakpoint	Largura	Comportamento
Mobile	< 640px	Stack vertical: header empilhado, sidebar em cima, mapa em baixo. Input + botão empilhados. Badge reduzido para "CDRView". Scroll horizontal nas tabs.
Tablet	640px — 1023px	Header horizontal. Sidebar + mapa ainda empilhados. Cards em grid de 2 colunas.
Desktop	≥ 1024px	Split view: sidebar 40% à esquerda com scroll, mapa 60% à direita ocupando <code>calc(100vh - 96px)</code> . Tabs sem scroll.

Detalhes de implementação:

- Header: `flex-col sm:flex-row` — empilha em mobile
- Main: `flex-col lg:flex-row lg:h-[calc(100vh-96px)]` — split em desktop com altura total
- Sidebar: `lg:flex lg:flex-col lg:h-full lg:overflow-hidden` + `lg:flex-1 lg:overflow-y-auto` no conteúdo
- Mapa: `lg:flex lg:flex-col lg:h-full` + `flex-1` no container
- Tabs: `overflow-x-auto` + `w-max` para scroll horizontal em mobile
- Tooltip do mapa: `fixed` em mobile (colado ao fundo), `absolute` em desktop (centrado)
- Botões: `min-h-[44px]` para touch targets acessíveis
- Textos: `text-xs sm:text-sm`, `text-2xl sm:text-3xl`, etc.

8. Guia de Estilo

Cores institucionais:

Token	Hex	Uso
Fundo app	#F8F9FA	Background principal (cinza ultra-claro)
Azul tecnológico	#00B4D8	Botão "Consultar IA", borda do bloco resposta, indicador Formações
Verde dataset	#16a34a (green-700)	Badge "Dataset Visent CDRView Ativo"
Vermelho alerta	#E63946	Círculo de alerta no mapa, barra de progresso baixa
Laranja parcial	#F4A261	Cobertura parcial no mapa
Verde sólido	#06D6A0	Infraestrutura sólida no mapa

Cores dos 5 serviços:

Serviço	Cor
Formações	#00B4D8
Empregabilidade	#2A9D8F
Exp. Estruturantes	#8B5CF6
Mentorias	#F4A261
Saúde Mental	#E76F51

Tipografia:

- Fonte: system-ui (padrão do sistema)
- Pesos: `font-normal` (400), `font-semibold` (600), `font-bold` (700)
- Tamanhos: `text-xs` (12px) a `text-xl` (20px) com variações responsivas
- Cor do texto escuro: `text-gray-800` (#1f2937)

- Cor do texto claro: `text-gray-400` / `text-gray-500`

Sombras e cantos:

- Cards: `shadow-sm` + `rounded-xl` (12px)
- Input: `shadow-sm` + `rounded-2xl` (16px)
- Botões: `rounded-2xl` (16px) ou `rounded-xl` (12px)
- Badges: `rounded-full`
- Header: `shadow-sm`

9. Fluxo do Utilizador

1. **Abrir a app** — vê o header, as tabs dos 5 serviços e o split sidebar + mapa
2. **Selecionar serviço** — clica numa tab (ex: Formações). O mapa atualiza cores e legendas
3. **Escolher região** — dropdown no topo da sidebar (opcional: "Todas as Regiões")
4. **Fazer pergunta** — clica numa sugestão OU digita no input
5. **Clicar "Consultar IA"** — vê spinner de 1.2s, depois aparece:
 - Bloco com a resposta em linguagem natural
 - 2 cards com valores e barras de progresso
 - Badges com as fontes dos dados
6. **Explorar mapa** — clica nos círculos para ver popups com indicadores
7. **Mudar idioma** — clica em PT | ES | EN no header. Todo o conteúdo traduz
8. **"Nova Consulta"** — limpa o resultado e volta ao estado inicial

10. Estado Actual vs Próximos Passos

✓ O que está feito (Semana 1 — Fundação mockada):

- ✓ Layout responsivo desktop/mobile com split 40/60
- ✓ Header + tabs dos 5 serviços + seletor de idioma
- ✓ Sidebar com input, sugestões contextuais, loading, resultado mockado
- ✓ Mapa Leaflet com 2 regiões, círculos coloridos, popups, legenda
- ✓ Cores dinâmicas no mapa conforme o serviço selecionado
- ✓ Tooltip fixo em mobile, popup em desktop
- ✓ Internacionalização PT/ES/EN completa
- ✓ Mock data por serviço (5 × respostas IA diferentes)
- ✓ Contratos API mockados (POST /dados, GET /mapa)
- ✓ Documentação técnica (este documento)

○ Próximos passos recomendados (Semana 2 — Integração real):

- ○ Configurar Vite + npm: `npm create vite@latest` + `npm install react-leaflet leaflet`
- ○ Ligar POST /dados ao backend real (substituir mock timeout por fetch)
- ○ Ligar GET /mapa ao backend real (substituir mock data por dados reais do PostgreSQL)

- Adicionar import `import 'leaflet/dist/leaflet.css'` no entry point do Vite
- Adicionar mais regiões conforme o dataset Visent CDRView
- Validar contratos com o backend (Victor) e serviço Python (Hércules)

○ Melhorias futuras (Semana 3 — Acabamento):

- Exportação PDF dos resultados
- Alertas por limiar (ex: cobertura < 20% destaca automaticamente)
- Comparativo entre regiões
- Filtros por período
- Suporte multilíngue para mais idiomas se necessário
- PWA: manifest.json + service worker para instalação
- Testes unitários (Jest + React Testing Library)

11. Como Executar

Opção 1 — Abrir index.html (imediato, sem build):

Navegar até: C:\Users\[user]\Desktop\app-bit\
Abrir: index.html (duplo clique)
Requisito: Internet (carrega React + Leaflet + Tailwind via CDN)

Opção 2 — Vite + npm (quando o registry estiver acessível):

```
cd C:\Users\[user]\Desktop\app-bit\  
npm install  
npm run dev  
# Abre em http://localhost:5173
```

Opção 3 — Python HTTP server (alternativa local):

```
cd C:\Users\[user]\Desktop\app-bit\  
python -m http.server 8080  
# Abre em http://localhost:8080
```

12. Notas Técnicas

- **Leaflet CSS:** No `index.html` é carregado via `<link>` do CDN. No Vite será via `import 'leaflet/dist/leaflet.css'` no entry point.
- **react-leaflet vs Leaflet puro:** O `MapaInterativo.jsx` (JSX) usa `react-leaflet` (declarativo). O `index.html` (CDN) usa Leaflet vanilla com `useEffect` + `useRef` (imperativo). Ambos produzem o mesmo resultado.
- **Mock vs Real:** Toda a lógica de dados está nos ficheiros `SidebarConsulta.jsx` e `MapaInterativo.jsx`. Para ligar ao backend, basta substituir os `setTimeout` por chamadas `fetch` aos endpoints.
- **Coordenadas:** As regiões usam coordenadas reais de Luanda, Angola (dataset Visent CDRView).
- **Semântica:** O código usa `<header>`, `<main>`, `<section>` para acessibilidade.
- **Sem emojis:** Todo o UI é profissional sem ícones gráficos (apenas texto + SVG).

- **Ficheiros na raiz:** `index.html` é standalone. Os componentes React estão em `src/components/`.

Contacto: Paufer — Frontend Principal, Equipa 48

Repositório: github.com/wongola-bit/appbit (dataset original: Vísent CDRView)

Documento gerado em: Junho 2026

App BiT — Hackathon B2G · Todos os direitos reservados à equipa 48